

Effective dating — impact report

What changed in the schema, how much complexity it adds, and what users will see. All seven phases are built, tested and merged to master.

8 new tables (7 history + frozen returns)	6 migrations (ed1–ed3, ed5–ed7)	~3,900 backend lines	~1,300 frontend lines	629 tests passing
---	---	--------------------------------	---------------------------------	-----------------------------

In one paragraph

Facts that change over time (status, programme, study intensity, module status, fee status, location, and opted-in custom attributes) are now stored as dated periods instead of being overwritten. The existing columns on the student still hold today's value, so lists, search and dashboards are unchanged. The HESA return now reads what was true in each period, and changes that reach a signed-off return are flagged and need the returns team. Phase 7 adds the HESA snapshot: a return built as at a date and as known at an exact moment, frozen at sign-off so late changes can't bleed into it.

The seven phases

Phase	What it delivered
1	Status history; writing up / withdrawal / termination events; direct status edits closed
2	Programme and study-intensity history; day-weighted STULOAD; return reads history
3	Module enrolment dates and dated module status; suspension proposes module interruption
4	Funding and supervision by period; child lists (status, modules, funding, supervisors)
5	Retrospective-change warnings; Dated history tab; as-of view; <code>returns.amend</code> rule
6	Fee status and location as dated facts; opt-in history for custom attributes
7	HESA snapshot: as at a date, as known at a moment, return frozen at sign-off

1. Schema changes

New tables (8)

Each stores a value with `valid_from` / `valid_to`, so we can see what was true on any day. Each also records who entered it and when, and whether it was later corrected. All are tenant-scoped with row-level security and a Postgres guard against overlapping periods.

Table	What it tracks	Phase
<code>student_status_history</code>	active / suspended / writing up / withdrawn ...	1
<code>student_programme_history</code>	Which programme, over time	2
<code>student_intensity_history</code>	FTE % (drives study mode and HESA STULOAD)	2
<code>module_enrolment_status_history</code>	Each module's status over time	3
<code>student_fee_status_history</code>	home / overseas / channel islands / unknown	6
<code>student_location_history</code>	Location of study	6
<code>student_custom_value_history</code>	Custom attributes that opt in to history	6

Table	What it tracks	Phase
report_return_version	The exact HESA return frozen at each sign-off (as at / as known at)	7

Changes to existing tables

Table	Change
module_enrolment	start_date / end_date (HESA module instance dates)
student	fee_status / study_location (today's value, kept in sync)
student_custom_field	track_history (opt-in flag, one-way)
All 7 history tables	closure flag (Phase 7): ending a period writes a closed copy instead of editing it, so the data can be rebuilt exactly as known at any moment
Funding / supervision	Already stored date ranges. Phase 7 adds ended_at (when the period was ended), so a snapshot knows whether it was still open then.
Permissions	New returns.amend (granted to Institution Administrator and dev only)

Unchanged by design

The existing columns on student (status, programme_id, study_mode) stay. They now hold today's value, kept in sync from the history tables, so every list, search and dashboard keeps reading one column — no joins, no change in speed.

Each migration fills in history for existing students from current data, so nobody starts with an empty history, and checks that the current values match it. There is no ed4 (Phase 4 needed no schema change). Every migration was rehearsed on a throwaway copy of the database, including downgrade and re-upgrade.

HESA snapshot (Phase 7)

Covers Demo 2 item 1.12: HESA returns are produced as a snapshot, and changes entered afterwards must not bleed in. Two independent times are involved:

Parameter	Question it answers	Example
As at (asAt)	Which value was in force on this date?	Live in August, Interrupted in March, back in June: as at 1 Aug the student is active; as at 1 Apr suspended
As known at (knownAt)	What did the system hold at this exact moment?	A suspension entered after sign-off but back-dated into the year is left out of the return as known at sign-off

- **Frozen at sign-off:** signing off builds the return as known at the sign-off moment (optionally as at a chosen date) and stores the exact rows. It re-downloads unchanged, whatever happens to the data later.
- **Left out of a snapshot:** history recorded later (even back-dated), funding or supervision ended later (still open then), students enrolled later.
- **Limit:** rows written before Phase 7 were closed in place, and older funding / supervision end times come from their last update. Exact rebuilds cover changes from now on.
- **Not in this phase:** amending and resubmitting a signed-off return (item 1.11) and XML output (item 1.10).

2. How much complexity it adds

Area	Size	Notes
Backend	~3,900 lines, 52 files	Most of it is one shared engine (<code>fact_history.py</code>) that every dated fact reuses. A new dated fact later is ~30 lines plus a migration.
Tests	~1,900 lines, 13 files	Golden-return, invariant, snapshot, migration and HTTP tests.
Frontend	~1,300 lines, 22 files	Mostly new components; existing screens gained date fields.

New rules developers must follow

- 1 One write path per fact.** Never update `student.status`, `programme` or `mode` directly; go through the history service. The generic student edit endpoint now refuses those fields with a 409.
- 2 Dates are half-open.** `valid_to` is the first day the value is no longer true. HESA and the screens show the inclusive end date (the day before).
- 3 Future-dated changes** are written immediately; the daily scheduler moves the current value on the day itself.
- 4 Nothing is edited in place or deleted.** A wrong row is superseded by a corrected one; a period that ends is superseded by a closed copy. Both stay for audit and exact snapshots.

```

period      [valid_from, valid_to)  valid_to = first day no longer true; NULL = still true
live row    superseded_by IS NULL (closure = true: a closed copy, not a change)
today's value student.status / programme_id / ... = live period covering today
              (written only by the history service)

```

What it costs at runtime

- A few extra rows per student per year.
- Screens cost nothing — they read the current-value columns.
- Building the HESA return got simpler and faster: ~700 students in ~0.7s, with no per-student replay of lifecycle events.

Where the risk is

Risk	Guard
Tests need a fixed “today” (a little harder to write)	A test helper sets the clock (<code>fact_history.today</code>).
SQL that bypasses the history service lets current values drift from history	Postgres overlap guard on every history table; each migration checks current values against history.

3. Frontend changes

Where	What changed
Student record — new Dated history tab	Timeline of every dated change (who recorded it, when, whether it was a correction), plus an “as of” date picker showing the record as it stood on any past day.
Student record — Record tab	New “Fee status and location” section with a dated Change form.
Lifecycle (suspension etc.)	Request form warns when a date reaches a signed-off return; pending rows and the Approve dialog show the same warning. A suspension can offer to interrupt the student’s open modules.
Funding	Start, change and end take a date (back-dating allowed, future not yet).
Supervision	Dated start and end (with a reason), plus a new Change supervisor action (same-day handover, same role).
Modules (taught)	Status changes take a date and reason; each module has its own start and end dates.
Statutory — sign-off tab	For a signed-off return: the students changed since sign-off, so registry can decide whether to resubmit; and Frozen returns — each signed-off version with its as-at date, the moment the data was known at, and a CSV download.
Custom attributes dialog	“Keep dated history” option; when on, entered data takes an effective date.
Statutory — sign-off dialog	Optional “Snapshot as at” date; the return is frozen as signed off.

Live check on every date field

As you type a date, the form checks whether it falls in a signed-off return. If you don’t hold `returns.amend`, it tells you to ask the student data / returns team; the server refuses the change either way.

4. What users will notice day to day

- **Most screens look the same.** The visible differences are date fields on change forms, the new tab, and the warnings.
- **Registry can no longer edit status, programme or mode directly.** They do it through lifecycle actions, which is the intended control.
- **Changes reaching a signed-off return need the returns team** (`returns.amend`). A year stays open to normal back-dating until its return is signed off. This is a process change ICR should be told about.

5. Status and open items

Item	Status
Phases 1–7	Built and merged to master (3abdae1)
Backend test suite	629 passed, 8 failed, 2 skipped. The 8 failures predate this work (assistant write intents, settings needing aiosmtplib, one pattern lab test).
Migrations	ed1–ed3 applied to the dev DB; ed5–ed7 rehearsed on a copy and apply on the next <code>start-backend.bat</code> .
HESA FEESTAT	Unmapped: needs confirmation of which HESA code each fee status maps to.
Study location	No data source yet; starts empty and is recorded per student.
Beyond effective dating	Still needed for HESA: catalogue versioning (CMA, item 1.3), unit of assessment (UOA), amend and resubmit (1.11), XML output (1.10), the Engagement entity (1.7).

Design and rationale: docs/approach/effective-dating/Effective_Dating_Approach.pdf.